Hoodies-Network: Business Rules and Semantics

How the Android HTTP library behaves, written as plain rules

How to read this

These are the rules the Hoodies-Network library follows, written in plain language. They come from the library source code, not from `MIGRATION_STRATEGY.md` or other design notes. Each rule names the file and class it comes from, so you can check it there.

- Rules are numbered by area, for example **R3** is rule 3 under *Retry and timeouts*.
- *Client* is a `HoodiesNetworkClient` made with its `Builder`. It holds the base URL, default headers and interceptors.
- *Request* is one call, such as `get` or `post`. It is a `Request` object placed on the shared request queue.
- *Interceptor* is app code that the client calls before a request is sent and after it succeeds or fails.
- *Cache entry* is a stored response body in the Room database `HoodiesNetworkCache`.
- *Cookie jar* is the cookie store a client can be given: in memory (`CookieJar`) or encrypted on disk (`PersistentCookieJar`).
- File paths are relative to `Hoodies-Network/src/main/kotlin/com/gap/hoodies_network/`.

---

1. Building the URL (U)

U1. The request URL is the base URL followed by the API path, with nothing added between them. The client joins the two strings as they are. It does not add or remove a `/`, so the base URL or the path must supply it.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getRequestRegular`, `getRequestFormUrlEncoded`: `validateUrl(baseUrl).plus(api)`).

U2. The base URL is lower-cased and given a scheme if it has none. The whole base URL is turned to lower case, even when it already starts with `http://` or `https://`. If it has no scheme, `https://` is added. A broken `https:/` or `http:/` prefix is repaired by adding the missing slash. An empty base URL is left empty, so the API path is then used as the full URL.

Source: `config/UrlResolver.kt` (`UrlResolver.validateUrl`).

U3. Query-parameter requests rebuild the URL from host and path, and drop a leading `www.`. `getUrlQueryParam` and `getUrlQueryParamEncoded` take the authority and path of the base URL and strip a leading `www.`. The scheme is kept only if it is `http` or `https`; any other scheme becomes `https`.

Source: `config/UrlResolver.kt` (`resolveUrl`, `getProtocol`, `resolveHttpProtocol`); `core/HoodiesNetworkClientNonInlined.kt` (`getStringRequest`, `getRequestUrlQueryParamEncoded`).

U4. Query values are percent-encoded, then the plain variant decodes them again. Parameters are added with Android's `Uri.Builder.appendQueryParameter`, which encodes them. `getUrlQueryParamEncoded` sends that encoded URL. `getUrlQueryParam` runs the whole URL through `URLDecoder.decode(..., "UTF-8")`, so it sends the raw characters.

Source: `utils/NetworkHelper.kt` (`getFinalURL`); `request/query/UrlQueryParamRequest.kt` (`convertToQueryParamURL`); `request/query/UrlQueryParamEncodedRequest.kt` (`convertToQueryParamEncodedURL`).

---

2. Request bodies and headers (B)

B1. The client offers five HTTP methods. `HoodiesNetworkClient.HttpMethod` has GET, POST, PUT, DELETE and PATCH. The lower-level transport also knows HEAD, OPTIONS and TRACE, but no client method uses them. An unknown method code fails with `IllegalStateException("request method type unknown")`.

Source: `core/HoodiesNetworkClient.kt` (`HttpMethod`); `connection/BaseNetwork.kt` (`setRequestMethod`).

B2. A body is only written for POST, PUT and PATCH. For GET and DELETE the body is never written to the connection, even when one is given.

Source: `connection/BaseNetwork.kt` (`setRequestMethod`, `applyBodyData`).

B3. Plain object bodies are turned into JSON with Gson. Any body passed as `Any` is sent as `gson.toJson(body)`. No body means an empty string. This includes `postRaw(api, String)`: the string is JSON-encoded, so it is sent with surrounding quotes. `JSONObject` and `JSONArray` bodies are sent as their `toString()`.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getRequestRegular`, `post(api, requestBody: Any, ...)`); `core/HoodiesNetworkClient.kt` (`postRaw`); `request/json/JsonObjectRequest.kt`, `JsonArrayRequest.kt`.

B4. Form encoding is chosen only by an exact `Content-Type` header on the call. If the call's additional headers have the key `Content-Type` with the value `application/x-www-form-urlencoded`, the body is sent as a form. The body must then be a `HashMap`; anything else becomes an empty form. Keys and values are URL-encoded in UTF-8 and joined with `&`.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getRequest`, `getRequestFormUrlEncoded`); `request/FormUrlEncodedRequest.kt` (`getBody`).

B5. A body without a content type is labelled `application/json`. When the request headers have no key `Content-Type` (exact spelling), the transport sets `application/json` before writing the body. Request bodies are always encoded as UTF-8.

Source: `connection/BaseNetwork.kt` (`applyBodyData`, `sendData`).

B6. Per-call headers win over the client's default headers. The client copies its default headers, then adds the call's additional headers on top, so a header with the same name is replaced.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getRequestRegular`: `headers.putAll(additionalHeaders)`).

B7. File uploads are multipart POSTs with numbered part names. `postMultipartFiles` always uses POST and sets `multipart/form-data; boundary=<boundary>`. The default boundary is a random UUID. Each file is a part named `file0`, `file1`, and so on, with its file name and a content type from `Files.probeContentType`. The number is the file's first position in the list, so a file listed twice gets the same name both times. The multipart content type is set before the additional headers, so a caller can override it.

Source: `core/HoodiesNetworkClient.kt` (`postMultipartFiles`); `core/HoodiesNetworkClientNonInlined.kt` (`getFileUploadRequest`); `request/Request.kt` (`postMultipartFormData`).

---

3. Sending and status codes (H)

H1. Only status codes 200 to 209 count as success. Any other code, including 3xx that were not followed and every 4xx and 5xx, is treated as a failure. A response code of `-1` is also a failure ("Could not retrieve response code from HttpUrlConnection.").

Source: `connection/BaseNetwork.kt` (`executeRequest`, `STATUS_OK = 200`, `STATUS_NOT_SUPPORTED = 209`).

H2. On an HTTP error, the error body becomes the error message. For codes of 400 and above the body is read from the error stream. The failure then carries that text as its message and the HTTP code as its code. A missing body is read as zero bytes, not as an error.

Source: `connection/BaseNetwork.kt` (`executeRequest`, `catch (e: IOException)`).

H3. Each connection disables HTTP caching and follows the global redirect setting. `useCaches` is set to false. Redirects follow `HttpURLConnection.getFollowRedirects()`, which is a process-wide setting.

Source: `connection/BaseNetwork.kt` (`openConnection`, `initConnection`).

H4. A custom TLS socket factory is used only for its own host. The factory is applied only when the URL is HTTPS and its host equals the configured `sslHost`. All other hosts use the default.

Source: `connection/BaseNetwork.kt` (`openConnection`).

H5. The connection is closed only if the request fails before the status check passes. Once a 200 to 209 status is seen, the connection is left open. This holds even if a later step fails, such as decryption, caching or cookie storage.

Source: `connection/BaseNetwork.kt` (`executeRequest`: `keepConnectionOpen`, `finally`).

H6. Response headers can be looked up in any letter case. Headers with a null name (the status line) are dropped. The rest go into a case-insensitive map where, for a repeated header, the last value wins; the full list is also kept.

Source: `core/Response.kt` (`toHeaderMap`, `getAllHeaders`); `connection/BaseNetwork.kt` (`convertHeaders`).

---

4. Interceptors (I)

I1. Interceptors run in the order they were added, before the request is sent. For each interceptor in turn, the client calls `interceptNetwork`, then `interceptRequest`. These calls happen on the caller's thread before the cache is checked or the request is queued.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`sendRequest`); `interceptor/Interceptor.kt`.

I2. The library does not block requests when the device is offline. `interceptNetwork` is told whether the device is online, but the request still goes ahead. Blocking is up to the interceptor. "Online" means the active network has Wi-Fi, cellular or Ethernet. A null context counts as offline.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`sendRequest`); `config/IsOnline.kt` (`isOnline`).

I3. An interceptor can cancel a request and choose its result. Calling `cancelRequest(result)` stops the request. The caller gets exactly that result, which may be a `Success` or a `Failure`. No network call or cache lookup is made. After a cancel, later interceptors still get `interceptNetwork` but not `interceptRequest`.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`sendRequest`); `request/CancellableMutableRequest.kt` (`cancelRequest`).

I4. `interceptResponse` sees every success before the caller does. It runs for each interceptor in order, then the caller is resumed. For a cache hit the request argument is null.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`successCallback`, `successCallbackImage`, `successCallbackHtml`).

I5. `interceptError` sees failures and can ask for a retry. It gets the error, the request and an attempt count. The count is 0 when automatic retry does not apply. If an interceptor calls `retryRequest()`, the request is queued again and the caller does not get this failure.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getFailureCallback`, `isRetryAttempts`); `request/RetryableCancellableMutableRequest.kt` (`retryRequest`).

---

5. Retry and timeouts (R)

R1. Automatic retry is off unless the client turns it on. `Builder.retryOnConnectionFailure` must be true, and the error's cause must be a `SocketException`, `SocketTimeoutException` or `IOException`. Because HTTP error codes are raised as `IOException` (rule H1), 4xx and 5xx responses are retried too, not just network drops.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`validateRetryRequest`); `core/HoodiesNetworkClient.kt` (`Builder.retryOnConnectionFailure`).

R2. The retry count is a fixed menu, and "max" means five. `RETRY_NEVER` = 0, `RETRY_ONCE` = 1, `RETRY_TWICE` = 2, `RETRY_THRICE` = 3, `RETRY_MAX` = 5. The default is `RETRY_NEVER`.

Source: `core/HoodiesNetworkClient.kt` (`RetryCount.parseToInt`, `Builder.retryOnConnectionFailure(..., maxRetryAttempts)`).

R3. Retries are sent again at once, with no delay or backoff. A retry puts the same `Request` object back on the queue straight away. The caller gets a `Failure` only after the number of failures is greater than the retry count.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getFailureCallback`, `isRetryAttempts`).

R4. The last failed attempt still queues one more send. The request is re-queued before the limit is checked. So when the limit is reached, the caller gets its `Failure`, but one more send has already been queued. The result of that send does not reach the caller, though interceptors may still see it.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`getFailureCallback`, `safeResume`).

R5. Requests are tracked by a millisecond timestamp. Each call's identifier is `System.currentTimeMillis().toString()`. Retry bookkeeping is keyed by it, so two calls started in the same millisecond on one client share a key. Image calls create a second identifier for their callbacks. If the two differ, `interceptResponse` gets `null` as the request.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`sendRequest` and siblings: `val identifier = System.currentTimeMillis().toString()`; `getImage`, `getImageRequest`, `successCallbackImage`).

R6. Timeouts are global and default to 60 seconds. Connect and read timeouts live in `HttpClientConfig`, shared by every client in the process. They are read each time a connection is opened. A value of 0 means no timeout. `setFactoryDefaultConfiguration()` puts both back to 60 seconds.

Source: `config/HttpClientConfig.kt` (`HttpClientConfig`); `connection/BaseNetwork.kt` (`openConnection`).

R7. Cancelling the caller's coroutine does not stop the HTTP call. The client does not register a cancel handler. A cancelled caller is simply not resumed when the result arrives.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`safeResume`: `if (continuation.isCancelled || continuation.isCompleted) return`).

---

6. Caching (C)

C1. Caching is off unless a call passes `CacheEnabled`. Every client method defaults to `CacheDisabled()`. The raw methods and file uploads always use `CacheDisabled()`.

Source: `core/HoodiesNetworkClient.kt` (method defaults, `postRaw`, `putRaw`, `deleteRaw`); `core/HoodiesNetworkClientNonInlined.kt` (`sendFileUploadRequest`).

C2. A cache entry is keyed by URL plus a hash of the body. The key is the full request URL and `String.hashCode()` of the request body. Two different bodies with the same hash share an entry.

Source: `cache/EncryptedCache.kt` (`cacheRequestResult`, `getCachedData`); `cache/persistentstorage/CacheDao.kt`.

C3. An entry is fresh until it is older than the threshold. The default threshold is one hour. An entry exactly at the threshold is still fresh; it goes stale only when its age is greater.

Source: `cache/EncryptedCache.kt` (`isDataStale`); `cache/configuration/CacheEnabled.kt` (`staleDataThreshold = Duration.ofHours(1)`).

C4. A fresh hit skips the network and looks like a plain 200. The cached body is delivered as status 200, with no headers and zero network time. The interceptors' `interceptNetwork` and `interceptRequest` have already run by then (rule I1).

Source: `cache/EncryptedCache.kt` (`getCachedData`); `core/HoodiesNetworkClientNonInlined.kt` (`sendRequest`).

C5. Every successful response is written to the cache in the background. When caching is on, any 200–209 response replaces the old entry for its key. The write runs on a background coroutine. The body is stored after the encryption interceptor has decrypted it (rule E3).

Source: `connection/BaseNetwork.kt` (`executeRequest`); `cache/EncryptedCache.kt` (`cacheRequestResult`).

C6. The cache has no size limit or clean-up. Entries are only replaced by a newer response for the same key. Nothing deletes old entries.

Source: `cache/EncryptedCache.kt`; `cache/persistentstorage/CacheDao.kt`.

C7. Encrypting cache entries is a per-call choice and is off by default. With `encryptionEnabled = true`, the body is encrypted with the keystore key (rule E4). On read, an entry is decrypted if it has a stored IV, whatever the current setting.

Source: `cache/configuration/CacheEnabled.kt` (`encryptionEnabled = false`); `cache/EncryptedCache.kt` (`cacheRequestResult`, `getCachedData`).

---

7. Encryption and keystore (E)

E1. The app can plug in one encryption interceptor for traffic.
`Builder.addEncryptionDecryptionInterceptor` sets a single `EncryptionDecryptionInterceptor`. Setting it again replaces the old one. By default there is none.

Source: `core/HoodiesNetworkClient.kt` (`Builder.addEncryptionDecryptionInterceptor`); `interceptor/EncryptionDecryptionInterceptor.kt`.

E2. Outgoing bodies, query parameters and extra header values are encrypted. The whole UTF-8 body is passed to `encryptRequest` before it is written. For query-parameter requests, each key and each value is encrypted. Only the values of the call's additional headers are encrypted; header names and default headers are not. The caller's header map is changed in place. File upload contents are not encrypted.

Source: `request/Request.kt` (`getBody`); `core/HoodiesNetworkClientNonInlined.kt` (`handleUrlQueryParamEncryption`, `handleAdditionalHeaderEncryption`).

E3. Successful responses are decrypted before anything else sees them. `decryptResponse` runs on every 200–209 body before it is cached or parsed. Error bodies are not decrypted.

Source: `connection/BaseNetwork.kt` (`executeRequest`).

E4. Data at rest uses one AES-GCM key in the Android Keystore. The key alias is `HoodiesNetworkCacheKey`. The cipher is `AES/GCM/NoPadding` with a 128-bit tag. The key is made on first use, then kept in memory. The same key protects the cache and persistent cookies.

Source: `keystore/CacheKeyManager.kt` (`getKey`, `generateNewKey`, `keyAlias`); `cache/EncryptedCache.kt` (`runAES`).

E5. The library makes its own 12-byte IVs and checks they are new. Keystore-side random IVs are turned off, so the library supplies a 12-byte IV from `SecureRandom`. For cache and cookie rows it draws again until the IV is not already in the database. (A code comment says 16 bytes; the code uses 12.)

Source: `keystore/CacheKeyManager.kt` (`generateNewKey`); `cache/EncryptedCache.kt` (`genIV`, `cacheRequestResult`); `cookies/persistentstorage/EncryptedDaoWrapperForCookies.kt` (`insert`).

---

8. Cookies (K)

K1. Cookies are off unless the client gets a cookie jar. Without `Builder.enableCookiesWithCookieJar`, no cookies are sent or stored.

Source: `core/HoodiesNetworkClient.kt` (`Builder.enableCookiesWithCookieJar`); `connection/BaseNetwork.kt` (`addStoredCookiesToRequest`).

K2. Stored cookies replace any `Cookie` header on the request. Before sending, if the jar returns cookie lines for the URL, they are joined with no separator added and set as the `Cookie` header.

Source: `connection/BaseNetwork.kt` (`addStoredCookiesToRequest`).

K3. Cookies are saved only from successful responses. `Set-Cookie` headers are passed to the cookie jar only after the status passes 200–209. Cookies on error responses are dropped.

Source: `connection/BaseNetwork.kt` (`executeRequest`: `request.cookieManager?.put(...)`).

K4. `CookieJar` is in memory; `PersistentCookieJar` is encrypted on disk. `CookieJar` wraps `java.net.CookieManager` with a null policy, so Java's default policy applies. `PersistentCookieJar` uses a Room database named after its `instanceName`. Every cookie is stored as Gson JSON, encrypted with AES-GCM (no switch to turn this off).

Source: `cookies/CookieJar.kt`; `cookies/PersistentCookieJar.kt`; `cookies/persistentstorage/EncryptedDaoWrapperForCookies.kt`.

K5. Persistent cookies are shared across HTTP and HTTPS, and port and path are ignored. The storage key is the scheme with `https` changed to `http`, then `://` and the host.

Source: `cookies/persistentstorage/EncryptedDaoWrapperForCookies.kt` (`uriToHost`).

K6. The persistent store never de-duplicates or expires cookies. `add` always inserts a new row, and `get` returns every row for the host. `remove` ignores the URI and deletes every row matching the cookie's `hashCode()`.

Source: `cookies/persistentstorage/PersistentCookieStore.kt`; `cookies/persistentstorage/EncryptedDaoWrapperForCookies.kt` (`insert`, `getByHost`, `deleteCookie`).

K7. Cookie helpers work per host, not per path. `getCookiesForHost` returns all cookies for a host. `setCookiesForHost` removes them first, then adds the new list. `removeAllCookies` clears the jar.

Source: `cookies/CookieJar.kt` (`getCookiesForHost`, `setCookiesForHost`, `removeAllCookies`).

---

9. Threading and delivery (D)

D1. All clients share one request queue with four worker threads. `RequestQueue` is a process-wide singleton. It starts four `QueueHandler` threads at background priority.

Source: `connection/queue/RequestQueue.kt` (`getInstance`, `DEFAULT_NETWORK_THREAD_POOL_SIZE = 4`); `connection/queue/QueueHandler.kt`.

D2. The queue has no real priority. It is a `PriorityBlockingQueue`, but `Request.compareTo` always returns 0, so no request is ranked above another.

Source: `connection/queue/RequestQueue.kt`; `request/Request.kt` (`compareTo`).

D3. The first queue creation fixes the TLS settings. `sslHost` and `sslSocketFactory` are taken only when the singleton is first built. The client uses `RequestQueue.instance`, which passes none.

Source: `connection/queue/RequestQueue.kt` (`getInstance`, `instance`); `core/HoodiesNetworkClientNonInlined.kt` (`requestQueue = RequestQueue.instance`).

D4. Network work and parsing happen on worker threads; results go to the main thread. A worker runs the HTTP call and parses the body. The result is posted through a `Handler` on the main looper. Cache hits are the exception: they are delivered from the background IO coroutine.

Source: `connection/queue/NetworkHandler.kt` (`executeRequest`); `connection/queue/RequestQueue.kt` (`init`: `Handler(Looper.getMainLooper())`); `core/HoodiesNetworkClientNonInlined.kt` (`sendRequest`: `Dispatchers.IO`).

D5. Only library errors are turned into failures on the worker thread. `NetworkHandler` catches `HoodiesNetworkError` only. `QueueHandler` catches only `InterruptedException`.

Source: `connection/queue/NetworkHandler.kt` (`executeRequest`); `connection/queue/QueueHandler.kt` (`run`).

D6. The caller is resumed at most once. `safeResume` returns without doing anything if the coroutine is already cancelled or completed.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`safeResume`).

---

10. Results and errors (F)

F1. Every call returns `Success` or `Failure`. `Success` holds the value, the response headers and the request URL. `Failure` holds a `HoodiesNetworkError` with a message and a code.

Source: `core/Result.kt`; `core/HoodiesNetworkError.kt`.

F2. The result type decides how the body is converted. `Unit` gives `Unit`. `String` or `Object` gives the body as text. Any other type is parsed with Gson. A Gson syntax problem gives code -3; any other conversion error gives code -4.

Source: `core/HoodiesNetworkClientNonInlined.kt` (`successCallback`); `core/HoodiesNetworkErrorDescription.kt`.

F3. Failures before a status code have code -1. Timeouts carry the exception message. A bad URL gives `"Bad URL " + url`. After a status code is read, that code is used instead (rule H2).

Source: `connection/BaseNetwork.kt` (`executeRequest`, `UNKNOWN_ERROR_CODE`).

F4. Response text uses the `charset` from `Content-Type`, or UTF-8. If the response has no charset parameter, the body is read as UTF-8.

Source: `header/HttpHeaderParser.kt` (`parseCharset`); `request/StringRequest.kt` (`parseNetworkResponse`).

F5. JSON parse errors have their own codes. Bad JSON gives code -6, an unsupported encoding -5, and a null pointer -7. String responses use the same -5 and -7 codes.

Source: `utils/NetworkHelper.kt` (`getParseNetworkResponse`); `request/StringRequest.kt` (`parseNetworkResponse`); `core/HoodiesNetworkErrorDescription.kt`.

---

11. Images (G)

G1. Images are fetched with GET and never made larger. With `maxWidth` and `maxHeight` both 0, the image is returned at its own size. Otherwise it is only scaled down.

Source: `request/ImageRequest.kt` (constructor `Method.GET`, `resizeBitmap`, `getResizedBitmap`).

G2. The scale type decides the target size. When both limits are set, `FIT_XY` fills the box and ignores the aspect ratio. `CENTER_CROP` keeps the ratio and covers the box. Other types keep the ratio and fit inside the box.

Source: `request/ImageRequest.kt` (`getResizedDimension`, `getResizeValueWithSpecifiedHW`).

G3. Images are decoded one at a time, in power-of-two steps. Decoding is guarded by a global lock. The sample size is the largest power of two not above the shrink ratio. Running out of memory gives a failure with code -2.

Source: `request/ImageRequest.kt` (`parseNetworkResponse`, `sDecodeLock`, `findBestSampleSize`).

---

12. Mock web server (M)

M1. The mock server ships in the main library and listens on port 6969 by default. It is built on `com.sun.net.httpserver.HttpServer`. `usePort` changes the port.

Source: `mockwebserver/MockWebServerManager.kt` (`Builder.port`, `usePort`).

M2. A method with no handler gets 405. For each request, `handleRequest` runs first. Then the handler for the method runs. If none is set, the server sends 405 “Method not allowed”. Methods other than POST, GET, PUT, DELETE, OPTIONS and PATCH get no handler response.

Source: `mockwebserver/WebServerHandler.kt` (`internalHandler`, `return405`).

Configuration constraints and defaults

Client builder (`core/HoodiesNetworkClient.kt`, `Builder`)

Setting	Default	Rule
`baseUrl()`	`""`	Lower-cased; `https://` added if no scheme (U2).
`addHeader` / `addHeaders`	none	Overridden by per-call headers (B6).
`addInterceptor`	none	Run in the order added (I1).
`addEncryptionDecryptionInterceptor`	none	Only one; a new one replaces the old (E1).
`retryOnConnectionFailure` flag	`false`	Must be true for any automatic retry (R1).
`maxRetryAttempts` (same call)	`RETRY_NEVER`	0, 1, 2, 3 or 5 retries (R2).
`enableCookiesWithCookieJar`	none	No cookies sent or stored without it (K1).

Per-call options

Setting	Default	Rule
`cacheConfiguration`	`CacheDisabled()`	Raw methods and uploads are never cached (C1).
`CacheEnabled.staleDataThreshold`	1 hour	Stale only when older than this (C3).
`CacheEnabled.encryptionEnabled`	`false`	Uses the keystore key when on (C7).
`postMultipartFiles` boundary	random UUID	Content type can be overridden by headers (B7).
`ImageRequest` scale type (short constructor)	`CENTER_INSIDE`	The client's `getImage` needs it given.

Source: `core/HoodiesNetworkClient.kt`; `cache/configuration/CacheEnabled.kt`; `request/ImageRequest.kt`.

Global and internal constants

Setting	Value	Rule
Connect timeout	60,000 ms	Global, 0 = none (R6). `config/HttpClientConfig.kt`
Read timeout	60,000 ms	Global, 0 = none (R6). `config/HttpClientConfig.kt`
Worker threads	4	Shared by all clients (D1). `connection/queue/RequestQueue.kt`
Success status range	200–209	Anything else fails (H1). `connection/BaseNetwork.kt`
Read buffer	1,024 bytes	`MAX_BUFFER_SIZE`. `connection/BaseNetwork.kt`
Default body content type	`application/json`	Only if no `Content-Type` (B5). `connection/BaseNetwork.kt`
Default charset	UTF-8	Bodies and responses (B5, F4). `header/HttpHeaderParser.kt`
Cache database	`HoodiesNetworkCache`	No eviction (C6). `cache/EncryptedCache.kt`
Keystore alias	`HoodiesNetworkCacheKey`	AES-GCM, 128-bit tag (E4). `keystore/CacheKeyManager.kt`
IV length	12 bytes	Checked for uniqueness (E5). `cache/EncryptedCache.kt`
Mock server port	6969	`usePort` to change (M1). `mockwebserver/MockWebServerManager.kt`

Error codes (`core/HoodiesNetworkErrorDescription.kt`)

Code	Meaning	Where it comes from
HTTP status	Server answered outside 200–209	`BaseNetwork.executeRequest` (H2)
-1	Unknown / no status code	Connection, timeout or URL failure (F3)
-2	Out of memory	Image decode (G3)
-3	JSON syntax	Gson conversion (F2)
-4	Other exception	Result conversion (F2)
-5	Unsupported encoding	JSON parse (F5)
-6	JSON error	JSON parse (F5)

Code	Meaning	Where it comes from
-7	Null pointer	JSON parse (F5)

Build (`Hoodies-Network/build.gradle.kts`)

Setting	Value
`compileSdk`	35
`minSdk`	28
`targetSdk`	32
JVM target	17